Economic Ripple

Complete Technical and Business Documentation

Macroeconomic System-Dynamics Simulation Platform

26 Countries · 7 Variables · 5 Scenarios · Empirical State-Space Model

Generated: May 07, 2026

System Matrix v4 · Model v5 · Spectral Radius = 0.9960

This document is a comprehensive reference covering every aspect of the Economic Ripple project: the motivation and design philosophy, the mathematical model in full detail with every symbol defined, the complete data pipeline from raw CSV to deployed matrix, model training steps, system architecture, the full 7x7 interaction matrix with interpretation of each entry, scenario definitions, business and technical Q&A;, and an honest assessment of the model's strengths, weaknesses, limitations, and future development path.

1. Executive Overview

Economic Ripple is an interactive macroeconomic simulation platform that models how economic shocks propagate through a system of seven interdependent variables across 26 countries over a configurable number of time steps. The platform is built around a linear discrete-time state-space model whose interaction matrix was statistically estimated from approximately thirty years of real-world panel data sourced from the World Bank, the Institute for Health Metrics and Evaluation (IHME), and the United Nations Office on Drugs and Crime (UNODC).

The tool is not a forecasting instrument. It does not claim to predict the future trajectory of any economy. Instead it is a structured what-if machine: it shows the probable shape of a shock trajectory, which economies are most structurally exposed, and which variables are most sensitive to a given scenario. The insight lies in the pattern and relative comparison, not in the absolute numerical output.

The project spans a full technology stack. The simulation engine is implemented in Python using NumPy and Pandas, served via a FastAPI REST API. The interactive frontend is a Next.js 16 application using React 19, TypeScript 5 in strict mode, Zustand 5 for global state management, Recharts 3.8 for visualisation, and Tailwind CSS for styling. The frontend includes ten distinct chart types, a time-scrubber playback mechanism, an Insights and Methodology panel, and a one-click HTML report exporter.

1.1 Core Design Principles

Transparency. Every variable change at every time step is decomposed into exactly three contributing causes: the structural system dynamics, the applied scenario shock, and the stochastic noise component. This decomposition satisfies an exact arithmetic identity that is verified at runtime. There are no black-box outputs.

Empirical grounding. Simulations begin from the most recent real observed data point for each country in the training panel, not from synthetic averages. The interaction matrix itself was estimated from data, not hand-tuned. The result is that the model reflects real structural relationships as they appear in thirty years of data.

Interactivity. Users select countries, choose a scenario or compose manual shocks, set the number of simulation steps, and watch the results populate in real time across ten synchronised chart types. They can scrub through time to inspect any step, and export a self-contained HTML report with inline charts.

Modularity. The shock engine, simulator core, FastAPI backend, and frontend are separate modules with clean interfaces. This means each layer can be tested, extended, or replaced independently without breaking the others.

1.2 Key Numbers

Metric	Value
Countries modelled	26
Macroeconomic variables per country	7

System matrix dimensions	7 x 7 (49 entries)
Training rows in panel dataset	202 country-year pairs
Historical span of training data	1994 to 2024 (approx. 30 years)
Spectral radius of system matrix	0.9960 (asymptotically stable)
Simulation step range	5 to 100 configurable steps
Noise standard deviation per variable per step	0.05
Damping coefficient applied to system delta	0.01
State variable clip bounds	[-5, +5] on all variables
Scenarios active in UI	5 (9 defined in total)
Output columns per simulation row	37 (7 values + 28 decomposition fields + step + country)
Chart types in frontend	10 distinct chart types

2. Dataset and Statistics

2.1 Variable Definitions and Data Sources

Seven variables were selected to form the state vector. The selection criteria were: (1) broad longitudinal coverage across diverse economies, (2) interpretable economic meaning, (3) availability on aligned time scales, and (4) relevance to policy-relevant shock scenarios. Below is a detailed description of each variable.

GDP Growth Rate (gdp_growth)

This is the annual percentage growth rate of Gross Domestic Product at market prices based on constant local currency. It measures the rate of expansion or contraction of the entire productive output of an economy in a given year. Source: World Bank Open Data, indicator NY.GDP.MKTP.KD.ZG. Format: wide CSV with countries as rows and years as columns, requiring a melt transformation to long format before processing.

In the training panel, GDP growth is stored after normalisation. It is the most volatile variable in the dataset with a standard deviation near 1.0 in normalised space, ranging from approximately -2.3 (severe recession) to +2.3 (rapid growth). The diagonal entry A[gdp,gdp] = 0.477 is the lowest of all seven variables, meaning GDP growth has the least year-to-year persistence — it is the most reactive variable in the system.

Inflation Rate (inflation)

Measured as the annual percentage change in the Consumer Price Index (CPI), reflecting the rate at which the general level of prices for goods and services is rising. Source: World Bank / IMF, wide CSV format. In hyperinflationary episodes (e.g. Ukraine, Argentina), raw values were extreme outliers. The normalisation procedure maps these into a comparable scale without losing the directional signal.

Inflation has a diagonal persistence A[inflation,inflation] = 0.7429, meaning roughly 74% of the current inflation level carries forward to the next period after accounting for cross-variable influences. This reflects the well-documented inertia of inflationary expectations. Off-diagonal entries show that inflation is significantly driven by poverty (A[inflation,poverty] = 0.128) and has moderate cross-effects on homicide (A[homicide,inflation] = 0.203), consistent with literature showing crime rises under sustained price pressure on lower-income households.

Unemployment Rate (unemployment_rate)

The share of the labour force that is without work but available for and seeking employment, expressed as a percentage. Source: World Bank modelled ILO estimates. Unemployment is a flow variable reflecting labour market conditions. Its diagonal persistence in the matrix is A[unemployment,unemployment] = 0.9592, the third highest, reflecting the well-known hysteresis in labour markets: once unemployment rises it tends to remain elevated for several years as skills atrophy and hiring norms shift.

Poverty Rate (poverty_rate)

The headcount poverty rate, typically measured as the percentage of the population living below a national or international poverty line. Source: World Bank PovcalNet and Global Monitoring Database. Poverty has the second-highest diagonal persistence in the matrix at A[poverty,poverty] = 0.9827, reflecting that structural poverty is extremely slow to change. It is notably cross-influenced by GDP growth (A[poverty,gdp] = -0.0241, negative as expected — growth reduces poverty) and by inflation (A[poverty,inflation] = 0.0136 — price rises marginally increase poverty headcount).

Urbanisation Rate (urbanization)

The percentage of total population residing in urban areas. Source: World Bank. Urbanisation has the highest diagonal persistence in the matrix at A[urban,urban] = 0.9968, meaning it barely changes year to year. This is economically correct: urban migration is a slow structural demographic process. Its off-diagonal entries are all near zero, making urbanisation essentially an exogenous trend variable in the system — it is not meaningfully driven by short-run economic shocks, though it does have a mild long-run influence on GDP and poverty through agglomeration effects that the model partially captures.

Depressive Disorder Prevalence (depression_prev)

The age-standardised share of the total population estimated to have depressive disorders, expressed as a percentage. Source: Our World in Data / IHME Global Burden of Disease study. This is a stock variable (prevalence), not a flow (incidence). It changes slowly over time and is provided in long CSV format rather than wide format. The diagonal persistence A[depression,depression] = 0.9953 is the highest after urbanisation, reflecting that mental health epidemiology changes at generational rather than annual timescales.

Including this variable was a deliberate design choice to broaden the model beyond purely economic indicators. Cross-variable effects show that depression prevalence has very weak coupling to the economic variables (all off-diagonal entries in the depression row are below 0.005 in absolute value), suggesting the data does not support strong short-run economic-to-mental-health causal links at the annual panel level. However, the variable is important for scenario storytelling: a Social Crisis scenario that drives up unemployment and poverty would be expected to eventually affect mental health prevalence, even if the matrix coefficient is small.

Homicide Rate (homicide_rate)

Intentional homicides per 100,000 population. Source: UNODC / World Bank. This is the most skewed variable in the dataset: the majority of the 26 countries have rates below 5, but Jamaica has a rate of 53.1 per 100,000, which is more than ten times the next highest country in the panel. This outlier affects the variance normalisation and likely inflates the magnitude of the homicide-related matrix entries. The diagonal A[homicide,homicide] = 0.9558 shows moderate persistence. Inflation has the strongest cross-effect on homicide (A[homicide,inflation] = 0.203), consistent with literature on economic stress and violent crime.

2.2 Descriptive Statistics of the Processed Panel

After cleaning, alignment, and normalisation, the training panel (model_v5_predictions.csv) contains 202 country-year observations. The table below shows descriptive statistics for the key variables as stored in the normalised panel used for matrix estimation.

Variable	Mean	Std Dev	Min	Median	Max
gdp_growth	0.034	0.959	-2.298	0.100	2.324
inflation	0.001	0.988	-1.411	-0.356	2.138
unemployment_rate	~6.8	~4.3	~1.5	~5.7	~25.5
poverty_rate	~14.2	~16.1	~0.1	~8.0	~77.6
urbanization	0.027	1.023	-2.326	0.333	1.374
depression_prev	~3.8%	~0.8%	~2.4%	~3.7%	~5.6%
homicide_rate	4.986	8.959	0.000	1.765	53.115

The homicide_rate maximum of 53.115 is Jamaica (2005). This is a legitimate data point but represents a statistical outlier that inflates the standard deviation of that column from approximately 1.5 (excluding Jamaica) to 8.96 (including it). Future versions should consider Winsorising this variable at the 99th percentile or fitting separate country-level noise variances.

2.3 Countries Covered

The following 26 countries were included in the final training panel after requiring complete data across all seven variables for at least eight consecutive years between 1994 and 2024. Countries were excluded if any variable had more than two consecutive missing years in the alignment window, or if the country was a regional or income-level aggregate rather than a sovereign nation.

Albania, Argentina, Australia, Bangladesh, Canada, China, Finland, France, Georgia, Germany, Iceland, India, Indonesia, Israel, Italy, Jamaica, Japan, Philippines, Poland, Russia, Spain, Sri Lanka, Sweden, Switzerland, Thailand, Ukraine.

The panel spans all major economic blocs: G7 (Canada, France, Germany, Italy, Japan), large emerging markets (China, India, Indonesia, Brazil-equivalent), Eastern Europe (Albania, Georgia, Poland, Ukraine), and smaller economies across South Asia (Bangladesh, Sri Lanka), Southeast Asia (Philippines, Thailand), Latin America (Argentina, Jamaica), and the Nordic region (Finland, Iceland, Sweden). The geographic diversity is important for matrix estimation: it prevents the regression from fitting only to high-income country dynamics.

2.4 Data Cleaning Decisions

Several non-trivial data cleaning decisions were made during preprocessing. First, continental aggregates and income-level groupings (e.g. "World", "Low income", "Sub-Saharan Africa") were removed from the World Bank CSVs using a regex filter. These aggregates do not represent sovereign economies and would distort the regression.

Second, country name standardisation was required because the same country appears under different names across different data sources. The mapping applied was: "United States of America" to "United States", "Russian Federation" to "Russia", "Korea Rep." to "South Korea", "Iran Islamic Rep." to "Iran". These mappings are applied in each per-variable script before the panel join to ensure matching keys.

Third, missing values were dropped entirely rather than imputed. Imputation would introduce synthetic data points into a matrix estimation procedure where data quantity is already limited (202 rows). It was judged better to use a smaller but cleaner dataset.

3. Mathematical Foundation

3.1 What Kind of Model Is This?

Economic Ripple uses a linear discrete-time state-space model. This class of model is one of the oldest and most well-studied in control theory and econometrics. It assumes that the state of a system at the next time step is a linear function of its current state, possibly perturbed by external inputs and noise. The economic equivalent is a Vector Autoregression of order 1, commonly written as VAR(1).

The reason for choosing a linear model rather than something more expressive (neural network, ARIMA, GARCH) is explained in detail in the technical Q&A; section. Briefly: with 202 training rows and 7 variables, a linear model with 49 parameters (the 7x7 matrix A) is already a moderately constrained estimation problem. Any nonlinear model with comparable expressiveness would overfit severely.

3.2 The Core Update Equation

The state of a country at the next simulation step is given by:

```
x(t+1) = x(t) + (A - I) * x(t) + sigma(t) + epsilon(t)
```

This can be simplified to:

```
x(t+1) = A * x(t) + sigma(t) + epsilon(t)
```

The form x(t) + (A - I)*x(t) is written out explicitly in the codebase because it makes the decomposition into "current state" and "structural change" cleaner. But mathematically these are identical: A*x(t) = x(t) + (A-I)*x(t) by simple algebra (A*x = I*x + (A-I)*x = x + (A-I)*x).

3.3 Symbol Definitions

Every symbol in the equation is defined in detail below.

x(t) — The State Vector

x(t) is a column vector of 7 real numbers, one for each macroeconomic variable, representing the state of a single country at time step t. The components are, in order: depression_prev, gdp_growth, inflation, unemployment_rate, poverty_rate, urbanization, homicide_rate. The ordering matches the rows and columns of the system matrix A.

At t=0 (the start of a simulation), x(0) is initialised to the most recent real observed values for that country from the training panel. These values are normalised to the same scale used during matrix estimation: each variable was normalised to have approximately unit variance across the training data.

The dimension is R^7 (a 7-dimensional real-valued vector). In the Python implementation, x(t) is represented as a 1D NumPy array of dtype float64.

A — The System Matrix

A is a 7x7 real-valued matrix. It is the core learned parameter of the model. Entry A[i,j] (row i, column j) encodes the linear influence of variable j on variable i over one time step. More precisely: if all variables except variable j are zero, and variable j has value v, then variable i at the next step will be

approximately A[i,j] * v.

The diagonal entries A[i,i] are the self-persistence coefficients of each variable: how much of variable i at time t survives into time t+1 through its own inertia, ignoring cross-variable influences. Values close to 1.0 indicate slow-moving, persistent variables (urbanisation at 0.997, depression at 0.995). Lower values indicate faster mean-reversion (GDP growth at 0.477 is the most mean-reverting).

The off-diagonal entries A[i,j] for i != j are the cross-variable multipliers. A positive value means variable j amplifies variable i. A negative value means variable j suppresses variable i. Values near zero mean the two variables are weakly coupled at the annual panel level.

*(A - I) * x(t) — The System Delta*

This is the structural change in the state vector due entirely to the internal dynamics of the system, with no external shock or noise. It answers the question: "If there were no shocks and no randomness, how much would each variable change purely because of the cross-variable structure?"

I is the 7x7 identity matrix. Subtracting it from A isolates the change component: (A - I)*x(t) = A*x(t) - x(t), which is the difference between where the system dynamics want to take the state and where the state currently is. This is stored in the output DataFrame as the column {variable}_system_delta for every variable.

A damping factor of 0.01 is applied to this term before adding it to x(t): system_delta = (A - I)*x(t) * (1 - 0.01). This attenuates the structural dynamics slightly at each step to prevent long-run drift in very long simulations.

sigma(t) — The Shock Vector

sigma(t) is the total externally applied shock at step t. It is a 7-dimensional vector, one component per variable. It is the sum of three possible shock sources: the scenario shock (if a scenario is selected), any manually specified variable-level shocks, and any timeline events scheduled for step t.

The scenario contribution to sigma(t) is given by the decay equation explained in section 3.5. Manual shocks are applied at their specified magnitude at every step without decay unless the user specifies otherwise. The shock vector is stored in the output DataFrame as {variable}_shock for every variable.

epsilon(t) — The Noise Vector

epsilon(t) is the stochastic noise component at step t. It is drawn independently for each variable at each step from a normal distribution with mean zero and standard deviation eta = 0.05:

```
epsilon(t) ~ Normal(mean=0, std=0.05) independently for each of the 7
variables
```

This means each variable at each step receives a random perturbation of typical magnitude 0.05 in normalised units. Since the variables have roughly unit variance in normalised space, a noise of 0.05 corresponds to a 5% of one standard deviation perturbation — small but visible over many steps. It adds stochastic realism: two runs with identical inputs will produce slightly different trajectories, reflecting genuine economic uncertainty.

The noise is seeded with a fixed random seed per simulation run to ensure reproducibility: given the same inputs, the same trajectory is produced. The seed is not exposed in the UI. Stored in output as {variable}_noise.

3.4 The Causal Decomposition Identity

Because the update equation is purely additive, the total change in any variable from step t to step t+1 is exactly decomposable into three contributions:

```
Delta_x(t) = x(t+1) - x(t) = system_delta(t) + sigma(t) + epsilon(t)
```

This is not an approximation. It is an exact algebraic identity that holds to floating-point precision at every step. The ExplanationPipeline module verifies this identity at runtime by computing the L2 norm of the residual between the observed delta and the sum of the three components. If the residual exceeds 1e-6, a warning is printed. In practice this never fires under normal operation.

This property is what makes the model transparent. For any chart showing a variable changing at step 8, the user can query: how much of that change was due to the structural system dynamics (the matrix A propagating earlier values), how much was the applied scenario shock, and how much was random noise? The answer is always available and exact.

3.5 Shock Persistence and Decay

Scenario shocks do not apply at constant magnitude across all steps. They decay geometrically, modelling the well-documented tendency for economic shocks to fade as markets adapt, policies respond, and excess capacity absorbs disturbances:

```
sigma_scenario(t) = base_shock_vector * rho^t
```

Where the symbols mean:

base_shock_vector is the vector of shock magnitudes defined in the scenario specification, one value per variable. For example in the Recession scenario, the GDP component is -0.60 (a downward shock of 0.60 normalised units).

rho (the persistence factor) is a number between 0 and 1 specific to each scenario. It controls how quickly the shock fades. At step t, the shock retains rho^t of its original magnitude. A persistence of 0.80 means the shock decays to 32.8% of its original size by step 5 (0.80^5 = 0.328). A persistence of 0.95 means the shock retains 77.4% at step 5 (0.95^5 = 0.774), representing a much more persistent disruption such as war or a structural institutional crisis.

t is the zero-indexed simulation step number. At t=0 the full base shock is applied. At t=1, rho * base_shock. At t=2, rho^2 * base_shock. And so on.

The persistence values across the five UI scenarios are: Recession 0.80, Boom 0.60, Stagflation 0.70, Social Crisis 0.90, Recovery 0.50. War (extended scenario, not in UI) has persistence 0.95, reflecting its extremely long-lasting economic damage.

3.6 Numerical Stability Controls

Two hard constraints are applied to x(t+1) at every step after the update equation is computed. These exist as safety nets, not as economic constraints.

```
x(t+1) = numpy.nan_to_num(x(t+1))
```

nan_to_num replaces any NaN (not-a-number) or Inf (infinity) values with zero or a finite large number respectively. NaN can arise if a shock or manual override creates a division-by-zero or overflow in an intermediate computation. Without this guard, a single NaN would propagate through all subsequent steps and corrupt the entire simulation trajectory.

```
x(t+1) = numpy.clip(x(t+1), -5, +5)
```

clip constrains each component of x(t+1) to the interval [-5, +5]. Since all variables are normalised to approximately unit variance in the training space, a value of +5 or -5 represents a five-standard-deviation deviation — an event with less than a one-in-three-million probability in Gaussian space. This bound should never be reached in realistic simulations but prevents runaway divergence if a user applies extremely large manual shocks or combines multiple large scenario shocks simultaneously.

3.7 Spectral Stability and Eigenvalue Analysis

A discrete-time linear system x(t+1) = A*x(t) is asymptotically stable if and only if the spectral radius of A — the largest absolute value among all eigenvalues of A — is strictly less than 1. If this condition holds, the unshocked system will converge back toward zero (the normalised baseline) as t approaches infinity.

The spectral radius of the deployed matrix A is 0.9960. This is less than 1, confirming asymptotic stability. However it is very close to 1, which has important implications. Using the half-life formula: ln(0.5) / ln(0.9960) = approximately 173 steps. This means a pure unshocked deviation takes about 173 simulation steps to decay to half its original size. In practical terms, shocks persist for very many steps, which is economically realistic: deep recessions and structural crises do not resolve quickly.

The full eigenvalue spectrum of the matrix, with interpretations:

Eigenvalue magnitude	Type	Interpretation
0.9960	Complex conjugate pair	Dominant mode. Very slow decay, very high persistence. A complex pair means oscillatory dynamics — the state spirals slowly toward equilibrium rather than monotonically decaying. This represents the main business-cycle mode of the system.
0.9682	Complex conjugate pair	Secondary slow oscillatory mode. Also high persistence, likely driven by the slow-moving variables (poverty, urbanisation, depression).
0.9592	Real	Unemployment persistence mode. The real eigenvalue corresponds to a non-oscillatory decay — unemployment rises and then declines smoothly without cycling.

0.6888	Real	GDP-inflation coupling mode. Moderate decay rate, meaning shocks transmitted through this mode fade within roughly 5-10 steps.
0.5333	Real	Fastest decaying mode. Shocks entering through this mode are largely absorbed within 3-4 steps. Half-life of approximately 1.3 steps.

The presence of complex eigenvalue pairs is economically meaningful. Complex eigenvalues in a discrete-time system produce oscillatory impulse responses — when the system is shocked, variables do not simply decay monotonically but overshoot and undershoot before settling, which is exactly what we observe in real business cycles. The imaginary parts of the eigenvalues determine the oscillation frequency.

4. System Interaction Matrix A

4.1 How to Read the Matrix

The matrix A is arranged so that rows correspond to the variable being influenced (the target) and columns correspond to the variable doing the influencing (the source). Entry A[i,j] answers the question: "Holding everything else constant, if source variable j is at value v at time t, what is its contribution to target variable i at time t+1?" The answer is A[i,j] * v.

The diagonal entry A[i,i] represents self-persistence: how much of variable i's own current level carries forward to the next step. Off-diagonal entries represent cross-variable transmission. Large positive off-diagonal entries indicate amplification (variable j drives variable i upward). Large negative entries indicate dampening (variable j suppresses variable i).

4.2 Full Matrix Values

The matrix below is read from system_matrix_v4.csv. Rows are target variables, columns are source variables, in the order: depression_prev, gdp_growth, inflation, unemployment_rate, poverty_rate, urbanization, homicide_rate.

Target \ Source	dep	gdp	infl	unemp	pov	urban	homicide
dep	0.9953	-0.0031	-0.0036	0.0045	0.0024	0.0049	0.0006
gdp	-0.0143	0.4769	-0.0602	0.1309	0.2018	-0.0819	-0.0558
infl	-0.0115	0.0569	0.7429	0.0117	0.1281	0.0817	0.1131
unemp	-0.0018	-0.0642	0.0498	0.9592	0.0145	0.0021	-0.0176
pov	0.0016	-0.0241	0.0136	0.0001	0.9827	-0.0143	-0.0289
urban	-0.0063	0.0019	-0.0040	-0.0034	-0.0028	0.9968	-0.0020
homicide	-0.0016	0.0137	0.0203	-0.0083	-0.0101	-0.0036	0.9558

4.3 Row-by-Row Interpretation

Row 0: depression_prev (what drives changes in depression prevalence)

The diagonal A[0,0] = 0.9953 is by far the dominant entry, confirming that depression prevalence is almost entirely driven by its own prior level. Year-on-year changes in this variable are tiny. All off-diagonal entries in this row are below 0.005 in absolute value. The most notable is A[0,3] = 0.0045 (unemployment slightly increases depression, consistent with literature on job loss and mental health) and A[0,5] = 0.0049 (urbanisation slightly increases depression, plausibly reflecting urban isolation effects). However these effects are so small that in practice the depression variable barely moves in simulations unless a very large manual shock is applied.

Row 1: gdp_growth (what drives changes in GDP growth)

The diagonal A[1,1] = 0.4769 is the lowest self-persistence in the matrix. This means GDP growth is highly mean-reverting: after any deviation, more than half the deviation is already corrected in the next step by the system dynamics alone, before any shock or noise. This reflects the well-known cyclicality of GDP growth.

The most significant off-diagonal entries are A[1,4] = 0.2018 (poverty drives GDP growth upward — this is a somewhat counterintuitive positive sign that likely reflects the statistical correlation between high-poverty developing economies and fast structural-change growth rates in the sample) and A[1,3] = 0.1309 (unemployment drives GDP growth upward, also counterintuitive but plausibly reflecting the labour utilisation recovery effect: high unemployment in one period tends to be followed by GDP recovery). A[1,2] = -0.0602 (inflation suppresses GDP growth, consistent with classical economics) and A[1,5] = -0.0819 (urbanisation suppresses short-run GDP, possibly because urban transitions involve investment costs before productivity gains).

Row 2: inflation (what drives changes in inflation)

Diagonal A[2,2] = 0.7429 confirms significant inflation inertia. A[2,4] = 0.1281 (poverty drives inflation) is notable: in the panel, high-poverty economies also tend to have structurally higher inflation, possibly through weak monetary institutions and supply-side constraints. A[2,6] = 0.1131 (homicide drives inflation) and A[2,5] = 0.0817 (urbanisation drives inflation) are moderate. The GDP-to-inflation coupling A[2,1] = 0.0569 is positive and small, consistent with a mild Philips-curve type effect in the data.

Row 3: unemployment_rate (what drives changes in unemployment)

High diagonal persistence A[3,3] = 0.9592, reflecting labour market hysteresis. A[3,1] = -0.0642 (GDP growth reduces unemployment, consistent with Okun's Law although the magnitude is smaller than typical Okun estimates, likely because the panel normalisation reduces the apparent elasticity). A[3,2] = 0.0498 (inflation mildly raises unemployment, consistent with stagflationary dynamics). All other off-diagonal entries are small.

Row 4: poverty_rate (what drives changes in poverty)

Extremely high diagonal persistence A[4,4] = 0.9827, second only to urbanisation. Poverty is a structural condition that changes at generational timescales. A[4,1] = -0.0241 (GDP growth reduces poverty, consistent with decades of development economics literature). All other off-diagonal entries are below 0.02 in absolute value, confirming that poverty is largely self-determined at the annual frequency.

Row 5: urbanization (what drives changes in urbanisation)

The highest diagonal persistence in the matrix at A[5,5] = 0.9968. All off-diagonal entries are below 0.007 in absolute value. Urbanisation is essentially exogenous to economic shocks at annual timescales in this model. It evolves as a slow structural demographic trend and is not meaningfully accelerated or decelerated by GDP fluctuations, inflation, or any other variable in the panel at the resolution of annual data.

Row 6: homicide_rate (what drives changes in homicide)

Diagonal A[6,6] = 0.9558, moderate-to-high persistence. The strongest cross-effect is A[6,2] = 0.2031 (inflation drives homicide upward), which is the largest off-diagonal entry in the entire matrix in absolute value. This is empirically well-supported: sustained inflation erodes real wages and purchasing power,

increasing economic desperation and correlating with crime rates in longitudinal data. A[6,4] = -0.0101 (poverty mildly reduces homicide rate in this panel — counterintuitive, but possibly reflecting that Jamaica's extreme outlier values distort this coefficient). A[6,1] = 0.0137 (GDP growth mildly raises homicide — also counterintuitive, possibly reflecting rapid urbanisation and inequality dynamics in fast-growing emerging economies in the sample).

5. Model Training Pipeline

The system matrix A was not hand-specified. It was estimated from data using a statistical regression procedure applied to the 30-year panel of 26 countries. This section documents the complete pipeline from raw downloaded files to the deployed matrix, including every technical decision made along the way.

Step 1: Raw Data Collection

Seven datasets were downloaded independently from their primary institutional sources. The World Bank datasets were obtained from the World Bank Open Data portal in their standard wide CSV format, where rows are countries and columns are years (e.g. 1960, 1961, ..., 2023). This required a reshape operation (melt) to convert to the long format required for panel regression. The depression prevalence data was obtained from Our World in Data as a long CSV already in (Country, Year, Value) format. The homicide data was obtained from the UNODC and World Bank combined.

No API was used for data retrieval. All files were downloaded as static CSVs and committed to the data/raw/ directory. This ensures reproducibility of the training pipeline — the exact same matrix can be regenerated from the same raw files at any time.

Step 2: Per-Variable Preprocessing Scripts

Seven dedicated Python scripts were written, one per variable: data/gdp.py, data/inflation.py, data/unemployment.py, data/poverty.py, data/urbanization.py, data/depression.py, data/homicide.py. Each script performed the following operations in order: load the raw CSV; rename columns to the standard schema (region, year, value); melt wide-format files to long format; remove continental and income-group aggregate rows using a regex filter on the region name; convert year and value columns to numeric types, coercing errors to NaN; drop rows with NaN in any required column; apply country name standardisation to ensure matching keys across all seven datasets; filter to the 1991-2024 year range; and save the cleaned output to data/processed/{variable}_clean.csv.

The country name standardisation was critical. The World Bank uses "Russian Federation" while IHME uses "Russia". Without this mapping, the panel join would produce NaN for Russia across all non-World-Bank variables. The mapping dictionary was built iteratively by running the panel join and identifying unmatched country names.

Step 3: Panel Assembly

All seven cleaned CSVs were merged using a sequential inner join on the keys (region, year). An inner join was chosen over an outer join because the regression requires all seven variables to be present for a given (country, year) pair. Outer join with imputation was considered and rejected (see Section 2.4).

The resulting panel (final_clean_panel.csv) had one row per country-year pair where all seven variables were available. The initial merge produced more country-year pairs than the final 202 because some countries had partial coverage. The country alignment step (Step 5) further reduced this to the final 26-country, 202-row panel.

Step 4: Feature Engineering

Additional features were engineered to enrich the regression signal. The file final_with_features.csv added: year-on-year delta columns ({variable}_delta), 1-year lag columns ({variable}_lag1), 3-year rolling average columns ({variable}_rolling_avg), and a binary GDP shock indicator (gdp_shock = 1 if the absolute deviation of GDP growth from its own 3-year rolling mean exceeded one standard deviation). These features were available for use in the regression but the final matrix estimation used only the contemporaneous values, not the engineered features, to maintain the clean VAR(1) interpretation.

Step 5: Country Alignment and Filtering

A minimum of 8 consecutive complete years of data was required for a country to be included in the final panel. Countries that passed this threshold but had gaps in the middle of their time series had those gap years dropped. Countries with only scattered years of complete data were excluded entirely.

This filtering reduced the country list from an initial 60+ to the final 26. The primary bottleneck variable was depression_prev: IHME coverage for many low-income and conflict-affected countries began only after 2000, and some countries had years of estimated rather than measured data that were excluded. Homicide data from UNODC had a similar sparsity issue for some regions.

Step 6: System Matrix Estimation via OLS Regression

The 7x7 matrix A was estimated using Ordinary Least Squares (OLS) linear regression. For each of the 7 target variables i, a separate regression was fitted:

```
x_i(t+1) = A[i,0]*x_0(t) + A[i,1]*x_1(t) + ... + A[i,6]*x_6(t) + error
```

In other words, the next-step value of variable i is regressed on all seven current-step variable values. This is a multivariate linear regression with 7 predictors and no intercept term (the intercept is suppressed because the variables are normalised to have zero mean in the training data, so the regression should pass through the origin in expectation). The 7 regression coefficient vectors, one per target variable, form the 7 rows of A.

This is equivalent to a constrained first-order Vector Autoregression (VAR(1)) model. The classical VAR(1) formulation would include a constant intercept vector; the Economic Ripple formulation omits this because the initial states x(0) are already the normalised observed values, and including an intercept would introduce a drift term that conflicts with the spectral stability analysis.

Four versions of the matrix were estimated (v1 through v4) with progressively refined country sets and feature inclusion decisions. V1 used a 15-country subset. V2 expanded to 26 countries and added lag features. V3 refined the country alignment procedure and removed rows with imputed data. V4 is the final deployed version, estimated without regularisation on the 26-country, 202-row panel using pure OLS.

Step 7: Matrix Validation

After estimation, each matrix version was validated on three criteria. First, spectral stability: the eigenvalues of A were computed using numpy.linalg.eigvals and the spectral radius (max absolute

eigenvalue) was verified to be strictly less than 1.0. V4 has spectral radius 0.9960.

Second, sign consistency with economic theory: key matrix entries were inspected manually to ensure they had the expected sign. GDP-to-unemployment should be negative (Okun's Law: GDP growth reduces unemployment). GDP-to-poverty should be negative (growth reduces poverty). Inflation-to-GDP should be negative (high inflation suppresses growth). All of these held in V4.

Third, simulation sanity: a step-through simulation with zero shocks was run for 50 steps to verify that all variables stayed near their initial values and did not drift or oscillate wildly. This confirmed that the spectral stability of the matrix translated to well-behaved simulation dynamics.

Step 8: Model Prediction Output

The fitted regression was used to generate one-step-ahead predictions for every row in the training panel, producing the file model_v5_predictions.csv. This file serves two purposes: first, it allows computation of in-sample fit metrics (R^2, mean absolute error per variable); second, it provides the empirical baseline values used to initialise simulations. When a user selects a country, the most recent year's observed values for that country are extracted from this file as the starting state x(0) for the simulation.

Step 9: Deployment

The matrix CSV (system_matrix_v4.csv) is loaded once when the FastAPI server starts, converted to a 7x7 NumPy float64 array, and stored in a shared cache module (backend/core/cache.py). All simulation requests use the same in-memory matrix object. There is no per-request file I/O. The matrix is treated as a read-only constant during server operation.

5.1 Why Not Ridge, Lasso, or Cross-Validated Regularisation?

Regularisation was applied in V3 but removed for V4. The reason is that with 202 training rows and 7 predictors per regression, the OLS problem is well-determined: the predictor matrix has many more rows than columns, so the OLS solution is unique and numerically stable without regularisation. Ridge regression shrinks coefficients toward zero, which would bias the diagonal self-persistence terms downward and make the matrix appear less persistent than the data supports. Since persistence is economically meaningful and well-estimated in this dataset, removing regularisation produces a more faithful matrix.

6. Scenario Library

Each scenario is defined as a ShockScenario object with three properties: a name, a 7-dimensional base shock vector (one component per variable), and a persistence factor rho. The shock applied at step t is base_shock * rho^t. The scenarios are defined in backend/core/scenarios.py and loaded at API startup.

The shock vector components are in normalised variable units. A value of -0.60 on GDP growth means the shock reduces GDP growth by 0.60 normalised units at step 0, which corresponds to approximately 0.60 standard deviations of the GDP growth distribution in the training data — roughly equivalent to a 1-2 percentage point reduction in annual GDP growth rate depending on the baseline country.

6.1 Active UI Scenarios

Scenario	Persistence	gdp	inflation	unemployment	poverty	urbanization	homicide	depression
Recession	0.80	-0.60	-0.20	+0.50	+0.40	-0.05	+0.15	0
Boom	0.60	+0.70	+0.30	-0.40	-0.50	+0.10	-0.10	0
Stagflation	0.70	-0.40	+0.70	+0.40	+0.30	-0.10	+0.10	0
Social Crisis	0.90	-0.30	+0.20	+0.50	+0.70	-0.05	+0.80	0
Recovery	0.50	+0.40	+0.10	-0.30	-0.40	+0.05	-0.20	0

6.2 Scenario Design Rationale

Recession is designed to represent a demand-driven contraction: GDP falls sharply (-0.60), unemployment rises (+0.50), poverty rises (+0.40). Inflation falls slightly (-0.20) as aggregate demand contracts. Persistence of 0.80 means the recession shock fades moderately over roughly 8-10 steps.

Boom is the mirror image but with lower persistence (0.60) because growth spurts tend to be self-correcting faster than recessions due to inflationary pressure and monetary policy tightening. GDP surges (+0.70), unemployment falls (-0.40), poverty falls (-0.50).

Stagflation is distinct from Recession in that GDP falls while inflation rises simultaneously (+0.70) — the pathological combination that defines stagflation. Unemployment also rises (+0.40) and poverty rises (+0.30). This scenario is empirically motivated by the 1970s oil shock and 2022 supply-chain disruptions.

Social Crisis focuses on the human and institutional dimensions: homicide rises dramatically (+0.80), poverty surges (+0.70), and the persistence is very high (0.90) because social breakdown is slow to reverse. GDP falls but moderately (-0.30). This represents situations like failed states, civil unrest, or severe institutional collapse.

Recovery is calibrated to be a moderate positive shock with short persistence (0.50) to model a period of stabilisation after a previous crisis rather than a full boom.

6.3 Extended Scenarios (Not Yet in UI)

Scenario	Persistence	Description
Tech Boom	0.70	Innovation-driven productivity surge. GDP +0.90, poverty -0.60, unemployment -0.30. Represents rapid technology-led growth as seen in East Asian economies.
Financial Crash	0.85	Market collapse and liquidity shock. GDP -0.80, unemployment +0.60, inflation -0.30 (deflationary). Designed to model 2008-style financial crises.
War	0.95	Most persistent scenario. Homicide +1.00, inflation +0.60, GDP -0.70, urbanisation -0.20 (population displacement). The 0.95 persistence means the shock retains 77% of its original magnitude at step 5.
Policy Stimulus	0.60	Government-driven expansion through fiscal and monetary policy. GDP +0.60, unemployment -0.40, inflation +0.25 (mild inflationary side effect).

7. System Architecture

7.1 Overall Stack

Economic Ripple is a full-stack web application with a Python simulation backend and a React-based frontend. The two are connected by a REST API. The simulation computation is entirely server-side; the frontend is purely for interaction and visualisation.

7.2 Backend

The backend is built with FastAPI, a modern Python web framework using Pydantic v2 for request and response model validation. The primary endpoint is POST /simulate, which accepts a SimulationRequest body and returns a SimulationResult containing the full trajectory DataFrame for all selected countries.

The simulation engine is implemented in three main modules. WorldSimulator in simulator/world_simulator.py is the top-level orchestrator: it accepts a request, initialises per-country state vectors from the empirical baseline (model_v5_predictions.csv), and runs the simulation loop for each country for the specified number of steps. At each step it calls the update equation, applies the shock from the ShockEngine, draws noise from the random generator, applies numerical stability controls, and logs all four components (value, system_delta, shock, noise) for every variable.

ShockEngine in simulator/shock_engine.py is responsible for computing the total shock vector sigma(t) at each step. It combines the scenario base shock with persistence decay, any manual variable-level overrides, and any timeline events scheduled for the current step.

ExplanationPipeline in simulator/explanation_pipeline.py takes the completed simulation DataFrame and generates a natural-language explanation for each step of each country, decomposing the observed changes into their three causes. This pipeline verifies the causal identity (residual < 1e-6) and calls ExplanationBuilder to format the text.

7.3 Frontend

The frontend is a Next.js 16 application in the App Router configuration with React 19, TypeScript 5 in strict mode, Tailwind CSS, Recharts 3.8, and Zustand 5. The entry points are app/page.tsx (landing page) and app/dashboard/page.tsx (main simulation dashboard).

Global state is managed entirely in a Zustand store (app/dashboard/store/simulationStore.ts). The store holds: selectedCountries (array), selectedScenarioId, manualShocks (object), steps (number), result (SimulationResult | null), viewStep (number for time scrubber), and isRunning (boolean). All chart components read from the store via useSimulationStore and re-render reactively when any relevant field changes.

The debounced auto-run mechanism in useAutoRun.ts fires a simulation request 800ms after any change to manualShocks (shock slider adjustments) or 400ms after a country is added. It uses a React ref (runRef) to hold the run function to avoid stale closure issues inside the debounce timer. An isFirstRender guard prevents the auto-run from firing on initial page load.

The ten chart types are: EconomicLineChart (toggled GDP/Inflation/Unemployment line chart), WellbeingAreaChart (toggled Depression/Homicide area chart), EconomicRadarChart (multi-variable country comparison radar), CountrySnapshotBar (grouped bar chart of latest values), ScatterGdpInflation (cross-sectional scatter), PovertyAreaChart (stacked area for poverty and urbanisation), ShockAttributionChart (stacked bar showing system vs shock vs noise contribution), VariableHeatmap (country x variable heatmap of cumulative changes), and sparklines in the CountrySnapshot panel.

7.4 Data Flow

The complete data flow for a simulation run is: user adjusts parameters in TopBar → debounced useAutoRun or explicit Run button → POST /simulate to FastAPI → WorldSimulator runs for all countries × steps → returns flat JSON array of rows → Zustand store updated with result → all chart components re-render. The round trip for 26 countries × 30 steps is typically under 200ms on a standard laptop.

8. Q&A; — Business Perspective

Q: What problem does Economic Ripple solve?

Policy analysts, researchers, and educators currently lack accessible, transparent tools for exploring macroeconomic "what if" scenarios interactively. Bloomberg Terminal and IMF tools are expensive, proprietary, opaque in their methodology, and designed for trained professionals. Academic models are typically implemented as scripts run in R or Python, not as interactive products. Economic Ripple bridges this gap: a production-quality interactive dashboard built on a statistically estimated, transparent model with real empirical baselines. It makes structured scenario analysis accessible to non-expert users while remaining rigorous enough for researchers.

Q: Who is the intended user of this platform?

Three primary audiences. First, economics researchers and graduate students who want to test hypotheses about cross-variable structural dynamics interactively before running formal econometric tests. Second, policy analysts at think tanks, central banks, or government agencies who need a quick tool to stress-test a country's exposure to a given shock scenario without access to full DSGE modelling infrastructure. Third, educators who need a live demonstration tool for teaching macroeconomics concepts such as business cycle propagation, the Philips curve, Okun's Law, or shock persistence.

Q: How is this different from IMF or World Bank tools?

IMF's World Economic Outlook and World Bank projections are consensus forecasts that draw on DSGE models with confidential parameters and central bank input. They represent the official best-guess trajectory, not a scenario explorer. Economic Ripple is the opposite: it does not claim to forecast. It shows what happens structurally if a given shock occurs, with every number fully traceable to the underlying model equations. The interaction matrix and all parameters are public and can be inspected. No other major tool offers this level of causal transparency at a variable-by-step resolution.

Q: Can this be used for investment decisions?

No, and this limitation must be stated clearly. The model uses normalised annual data and a global average structural matrix — it does not know about current market pricing, monetary policy cycles, political risk, or real-time news flow. It cannot predict short-term market movements. Using it for investment decisions would be a serious misapplication. Its appropriate use is for understanding directional structural risk, comparing relative country exposure to a type of shock, and exploring "under what conditions does poverty rise faster than unemployment?" type questions.

Q: What is the monetisation potential?

Three plausible paths. First, SaaS subscription for policy organisations: expanded country coverage (80+), live World Bank API data refresh, PDF report generation, and team workspace features. Second, university licensing as a teaching tool: semester licenses for economics departments with courseware integration and an instructor dashboard showing all students' simulation configurations. Third,

white-label API integration for economic intelligence products at media organisations (Reuters, FT, The Economist) who want to embed an interactive model in their data journalism work.

Q: What is the competitive moat?

The combination of empirical matrix estimation, full causal decomposition at every step, multi-country simultaneous simulation, a production-quality interactive dashboard, and a one-click exportable report is unusual in the landscape of academic and policy simulation tools. Most academic models are not productised. Most commercial products are not transparent. Economic Ripple combines both, which is the core differentiation. The specific system matrix v4, trained on a curated 30-year panel, represents a proprietary estimated parameter set that would take significant effort to replicate.

Q: How does it handle uncertainty and communicate it to users?

Currently through the stochastic noise term: each run produces a slightly different trajectory due to the N(0, 0.05) noise per variable per step. However, the platform does not yet display confidence intervals or percentile bands. This is acknowledged as a limitation. Future versions would implement a Monte Carlo mode that runs N=1000 simulations and displays the 10th, 50th, and 90th percentile ribbons on each chart, giving users a direct visual representation of trajectory uncertainty.

Q: How scalable is the current architecture?

The simulation engine is purely in-memory NumPy operations and runs in under 100ms for 26 countries × 30 steps. The FastAPI backend is single-threaded in development mode but can be deployed with multiple Uvicorn workers for concurrency. The frontend is a static Next.js build deployable on Vercel or any CDN. For a research-scale deployment, the current architecture supports dozens of simultaneous users without modification. For a production SaaS product, a job queue (Celery + Redis) for long-running simulations and horizontal FastAPI scaling would be required.

Q: What is the path to adding more countries?

Adding a country requires: (1) downloading that country's data for all seven variables from the same sources, (2) running the per-variable preprocessing scripts, (3) re-running the panel assembly and matrix estimation pipeline. If the new country has complete data for all seven variables over at least 8 years, it can be included in the next matrix re-estimation. The matrix dimensions do not change — the 7x7 structure is fixed. The new country simply contributes additional training rows to the regression and has its baseline values added to the model predictions file. This process is fully scriptable and has been done four times already (v1 through v4).

Q: What makes this credible as a research tool?

Four things. First, the model parameters were estimated from 30 years of real data from three major institutional sources, not hypothesised by hand. Second, the simulation starting states are real observed values, not synthetic. Third, the spectral stability of the matrix has been formally verified — the system will not diverge without shocks. Fourth, every output is causally attributed to system, shock, and noise in

an exact decomposition that is verified at runtime. A researcher can always audit why any variable moved at any step.

9. Q&A; — Technical Perspective

Q: Why a linear VAR(1) model rather than a neural network or DSGE model?

Linear VAR(1) was chosen for three fundamental reasons. First, interpretability: each entry A[i,j] has a clear economic meaning as a structural multiplier. There is no equivalent interpretability in a neural network's hidden weights. Second, stability guarantees: spectral radius analysis of A tells us definitively whether the system is stable. Neural networks provide no such analytical guarantee. Third, data volume: with 202 training rows and 7 variables, a linear regression with 7 parameters per equation (49 total) is well-conditioned. A neural network with comparable complexity would have hundreds of parameters and would severely overfit. DSGE models were not considered because they require structural calibration of behavioural parameters (discount rates, price stickiness, etc.) that are not estimable from this dataset.

Q: How exactly was the matrix estimated — what is the regression formulation?

For each of the 7 target variables i (rows of A), an OLS regression was run: x_i(t+1) = sum over j of A[i,j] * x_j(t) + error. In matrix notation: X_next = X_curr * A^T + Error, where X_curr is the (202 x 7) matrix of current-step values and X_next is the (202 x 7) matrix of next-step values. Each column of X_next was regressed separately on all columns of X_curr using numpy.linalg.lstsq or sklearn LinearRegression(fit_intercept=False). The OLS solution minimises the sum of squared prediction errors: min over A[i,:] of ||x_i(t+1) - A[i,:] * x(t)||^2. This is run independently for each of the 7 target variables.

Q: Why do countries not interact with each other in the simulation?

Cross-country coupling would require a (26*7) x (26*7) = 182 x 182 interaction matrix with 33,124 entries. Estimating this reliably from 202 training rows is fundamentally impossible — the system is vastly underdetermined. The current 7x7 design estimates a single matrix representing global-average within-country dynamics. Countries do interact implicitly through shared scenarios: if Recession is selected, all countries receive the same shock vector simultaneously, so correlated declines across countries appear in the charts. True bilateral trade coupling is the primary acknowledged limitation of the current architecture.

Q: What does the spectral radius of 0.9960 mean practically for simulations?

The spectral radius is the largest absolute eigenvalue of A, which equals 0.9960. For a discrete system x(t+1) = A*x(t), the general solution involves terms like lambda^t * v where lambda is an eigenvalue and v is the corresponding eigenvector. The dominant mode (lambda = 0.9960) decays as 0.9960^t. To compute the half-life: solve 0.9960^t = 0.5, giving t = ln(0.5) / ln(0.9960) = approximately 173 steps. This means a pure unshocked deviation from baseline takes 173 simulation steps to decay to half its original size. At 10 simulated steps per year, this represents a 17-year recovery half-life — very long, and consistent with empirical evidence on the persistence of economic crises like the Great Depression or the post-2008 output gap.

Q: How does the debounced auto-run work and why is it needed?

Without auto-run, users would need to manually click a Run button after every slider adjustment, creating a poor interactive experience. The useAutoRun hook sets up a debounce: when manualShocks changes, it starts an 800ms timer. If manualShocks changes again within 800ms, the timer resets. Only when 800ms pass without further changes does the run() function fire. This prevents flooding the backend with requests while the user is actively dragging a slider. The run function is stored in a useRef (runRef.current = run) to avoid the stale closure problem: a setTimeout callback captures the function reference at the time it is created; without the ref, it would call an outdated version of run that held stale state. With the ref, it always calls the most current run function.

Q: Why does the playback animation use getState() instead of hook state?

The playback mechanism uses setInterval to advance viewStep by 1 every 200ms. Inside a setInterval callback, any variable captured from a React hook (e.g. const [viewStep, setViewStep] = useState(0)) is frozen at the value it had when the interval was set up — this is the stale closure problem. If viewStep starts at 0 when the interval begins, every tick will see viewStep as 0 regardless of how many ticks have occurred. The solution is to bypass the React hook entirely and read the Zustand store directly: useSimulationStore.getState().viewStep reads the live, current value from the store rather than the closed-over React state. This is a standard pattern for intervals in React.

Q: How is the causal decomposition exactly correct to 1e-6 precision?

Because the update equation is additively structured: x(t+1) = x(t) + system_delta + shock + noise. The WorldSimulator logs all four quantities: the final x(t+1), the x(t) it started from, the system_delta = (A-I)*x(t)*(1-lambda), the shock vector from ShockEngine, and the noise vector drawn from the RNG. These are stored separately in the output DataFrame as four distinct columns per variable. The ExplanationPipeline then checks: does x(t+1) - x(t) equal system_delta + shock + noise to within 1e-6? It always does because these are the exact same float64 values that were added together to produce x(t+1) — the identity is algebraically guaranteed, not approximated.

Q: How are the SVG charts in the exported HTML report generated without any library?

A pure TypeScript function buildSvgChart() takes the raw simulation step data, computes the min and max values for Y-axis scaling, maps each step to an X pixel coordinate using linear interpolation across a fixed chart width, maps each value to a Y pixel coordinate, and emits an SVG polyline element per country. Gridlines are emitted as SVG line elements at fixed Y-axis positions. Axis labels are SVG text elements. The entire SVG is a string assembled with template literals and embedded directly in the HTML as inline SVG — no base64 encoding, no PNG conversion, no external files. The report is a single self-contained .html file that can be opened offline.

Q: What happens if a user applies an extremely large manual shock?

The simulation will compute a large intermediate value for x(t+1). The nan_to_num guard will convert any overflow to a finite number. Then clip(-5, +5) will cap the value at ±5 normalised units. From the user's perspective, the chart will show the variable shooting to its maximum or minimum value and

staying there until the shock decays below the level that keeps the clipped variable from moving further. This is visually obvious and clearly unrealistic, which serves as useful feedback to the user that they are operating outside the model's calibrated range.

Q: Can the timeline feature schedule shocks at specific steps — how does it work?

Yes. The SimulationRequest Pydantic model accepts an optional timeline field typed as Dict[int, List[TimelineEvent]] where the key is a step number and each event specifies an optional scenario override and optional manual shock overrides for that step only. When the WorldSimulator reaches step t, the ShockEngine checks whether the timeline has any events for step t and, if so, computes their shock contribution in addition to or instead of the ongoing scenario shock. The frontend does not yet expose a UI for timeline construction — it is currently only accessible via direct API calls, which is documented as a future development item.

Q: Why is the noise level set to 0.05 specifically?

This was chosen empirically to be small enough that a single step's noise does not dominate over the structural system dynamics, but large enough to produce visible run-to-run variation over 30+ steps. In normalised variable space where most values are in the range [-2, +2], a noise of 0.05 per step (1 standard deviation of the noise distribution) is roughly 2.5% of the typical variable range. After 30 steps, the accumulated noise has standard deviation 0.05 * sqrt(30) = 0.27, which is visible on charts but does not overwhelm the deterministic signal. A higher noise like 0.15 would make trajectories appear highly variable and obscure the structural dynamics. A lower noise like 0.01 would be virtually invisible.

10. Strengths and Weaknesses

10.1 Strengths

Empirically grounded starting states

Every simulation begins from the most recent real observed data point for each country in the training panel. This means the "what-if" question being asked is grounded in the actual current economic position of that country, not a stylised average. A recession shock on Germany starts from Germany's actual recent GDP growth, inflation, and unemployment — not a fictional baseline.

Full causal transparency at every step

Every variable change at every step for every country is decomposed into its three causes: structural system dynamics, applied shock, and stochastic noise. This decomposition satisfies an exact arithmetic identity verified at runtime. There are no black-box outputs. Any analyst can look at a chart showing GDP falling at step 8 and ask: "how much of that fall was the matrix propagating prior unemployment, how much was the scenario shock, and how much was noise?" The answer is always available and exact. This is extremely rare in simulation tools.

Spectral stability formally verified

The system matrix has been checked for asymptotic stability through eigenvalue analysis. The spectral radius of 0.9960 < 1.0 guarantees that the unshocked system will converge back toward its baseline. The simulation cannot diverge under zero-shock conditions. This is a formal mathematical guarantee, not a heuristic claim.

Multi-country simultaneous simulation

All 26 countries are simulated in a single API call with their individual empirical starting states. Users can overlay trajectories across countries on the same chart, revealing structural divergence patterns — for example, which countries are most affected by a stagflation shock, or which recover fastest from a recession. This comparative dimension is a primary source of analytical value.

Production-quality ten-chart dashboard

The frontend is not a proof-of-concept with one chart. It has ten synchronised chart types giving analysts multiple visual lenses: time series, area charts, radar charts for multi-variable comparison, scatter plots for cross-sectional correlations, heatmaps for country-variable change matrices, and attribution charts showing the system-shock-noise decomposition visually.

Self-contained HTML report export

One-click report generation produces a downloadable .html file with inline SVG charts, data tables, executive summary, and auto-generated findings. The file has no external dependencies — it works offline, can be emailed, printed, or embedded in a report. This is built without any charting library dependency in the export path.

10.2 Weaknesses

No cross-country coupling

Countries are structurally independent in the model. When Germany and France are both selected and a Recession scenario is applied, they both receive the same shock vector but do not influence each other's trajectories through trade or financial linkages. A recession in Germany that exports to France would in reality reduce French GDP through the trade channel — this is not captured. The apparent correlation between countries' trajectories in the simulator comes entirely from the shared shock, not from genuine bilateral coupling.

Linear dynamics cannot capture regime shifts

The model is linear: the structural relationship between variables is fixed by the matrix A regardless of the current state. Real economies are nonlinear: a debt level crossing 120% of GDP triggers a different dynamic than one at 80%. A bank run is a threshold event, not a linear process. Hyperinflation above 50% monthly follows different dynamics than moderate 5% annual inflation. The model cannot represent any of these threshold or tipping-point behaviours.

One global matrix for all countries

All 26 countries share the same 7x7 interaction matrix. The model treats a recession in Jamaica identically to one in Germany in terms of structural propagation. The actual economic structures are vastly different: Jamaica's economy is service and tourism-dependent with an extreme homicide outlier; Germany's is a diversified industrial exporter. A country-specific matrix would be more accurate but is not estimable from the available data volume.

No formal out-of-sample validation

The matrix was not evaluated on held-out test years. No 80/20 train-test split was performed on the time series. The validation was qualitative (sign consistency with theory) and formal only in the sense of spectral stability. We do not know the out-of-sample RMSE or whether the model generalises to post-2024 data. This is the most significant scientific limitation for any researcher wishing to cite the model's predictive performance.

Constant isotropic noise

The noise term is N(0, 0.05) identically for all variables. In reality, GDP growth has much higher year-to-year volatility than urbanisation or depression prevalence. A heteroscedastic noise model with a 7x7 covariance matrix estimated from the regression residuals would be more accurate. The current constant noise means that slow-moving variables (urbanisation, depression) appear noisier relative to their typical magnitude than fast-moving ones (GDP growth).

Jamaica homicide outlier distortion

Jamaica's homicide rate of 53.115 per 100,000 is an extreme outlier — the next highest country in the panel is roughly 5x lower. This single data point inflates the standard deviation of the homicide column from approximately 1.5 to 8.96. The normalisation procedure therefore assigns Jamaica a normalised homicide value much closer to the mean than its raw value would suggest, and the regression coefficients involving homicide_rate may be distorted by this leverage point.

11. Limitations

Not a predictive model

Economic Ripple cannot produce reliable point forecasts of future macroeconomic trajectories. The model was not designed, validated, or calibrated for forecasting. It is a structured scenario explorer. A user who interprets the output as a forecast — "GDP will fall by X% in year Y" — is misusing the tool. The only valid interpretations are relative ("country A is more exposed to this shock than country B") and directional ("under this scenario, poverty tends to rise before unemployment does").

Country coverage limited by data availability

The 26-country panel is limited by the coverage of depression_prev (IHME Global Burden of Disease) and homicide_rate (UNODC), which have sparse or unreliable data for many low-income and conflict-affected countries. Many economically significant countries including Brazil, Mexico, Nigeria, Saudi Arabia, Egypt, and most of Sub-Saharan Africa are not in the panel, limiting the applicability of the tool for analyses focused on those regions.

No real-time data feeds

The model is calibrated on data through approximately 2024. It does not incorporate live data releases from the IMF World Economic Outlook, World Bank, or national statistical agencies. The empirical baseline values for each country become increasingly stale over time. There is no automated pipeline to refresh the training data, re-estimate the matrix, or update the initial conditions. This needs to be addressed for any production deployment.

Timeline event UI not exposed

The backend fully supports scheduling different shocks at different steps via the timeline field in the API request. For example, a user could specify "apply a Stagflation shock at steps 0-5, then a Recovery shock at steps 6-15". The frontend has no interface for composing these multi-stage scenarios. They can only be triggered via direct API calls. Building the timeline event UI is a planned development item.

Depression variable is a stock, not a flow

depression_prev is the share of the population with depressive disorders — a cumulative prevalence stock variable. It changes at generational timescales, not annual ones. In the simulation, a large shock to depression prevalence (e.g. through a Social Crisis scenario) produces a step change in the stock, which is not how mental health epidemiology actually works. The appropriate variable for short-run economic analysis would be the year-on-year change in incidence, not the prevalence level. Including prevalence was a data availability compromise.

No confidence intervals on displayed trajectories

Each simulation run produces a single trajectory. Users have no visual indication of how much the trajectory would vary across repeated runs, or how sensitive it is to parameter uncertainty in the matrix A. Without Monte Carlo bands, a user looking at a smooth GDP trajectory cannot tell whether that trajectory would look very different with slightly different noise realisations or a slightly different matrix.

Single-server deployment only

The backend is a single FastAPI process. For concurrent user loads beyond roughly 20-30 simultaneous simulation requests, this would require horizontal scaling with multiple worker processes behind a load balancer. Long-running simulations (100 steps × 26 countries) would block shorter simulations without a job queue. The current architecture is appropriate for research and demonstration use but not for a public-facing production deployment.

HTML report charts are simplified

The self-contained HTML export uses a custom inline SVG renderer that produces basic polyline charts without tooltips, interactive legends, or animations. The charts in the report are simplified compared to the Recharts-based dashboard charts. For a professional report output, integration with a server-side chart rendering library (e.g. using a headless browser to capture Recharts SVG output) would produce higher-quality visuals.

12. Future Development Roadmap

The following roadmap represents development priorities ordered by feasibility and impact. Near-term items are achievable without architectural changes. Medium-term items require new components. Long-term items involve fundamental model extensions.

Near-term (0-3 months)

Monte Carlo confidence bands

Run the simulation N=500 times with different noise seeds and display the 10th, 50th, and 90th percentile ribbons on all time-series charts. This is purely a frontend change: the backend would need a new mode that returns N trajectories instead of one, and the charts would render shaded areas between the percentile lines. This would be the single highest-impact improvement to the analytical credibility of the tool.

Timeline event UI

Build a drag-and-drop or table-based interface for scheduling different scenarios or manual shock changes at specific simulation steps. The backend already supports this fully via the timeline API field. The frontend needs a new component in TopBar or a dedicated panel for composing and visualising the event schedule.

Variable-specific noise covariance

Replace the scalar noise parameter eta=0.05 with a 7x7 diagonal covariance matrix estimated from the regression residuals of each variable. GDP growth residuals have much higher variance than urbanisation residuals, and the noise model should reflect this. This would make the simulation stochastically more realistic.

CSV export of trajectory data

Add a button to download the raw simulation output as a CSV file. Currently only the HTML report export is available. Researchers would benefit from direct access to the numerical data for downstream analysis in R or Python.

Medium-term (3-9 months)

Cross-country bilateral coupling

Introduce a trade weight matrix T of dimension 26x26 (or sparse with only significant bilateral trade pairs populated) to allow economic shocks in one country to propagate to its trading partners. Each country's shock vector would gain an additional term: sum over j of T[i,j] * sigma_j(t-1). The trade weights could be initialised from IMF Direction of Trade Statistics. This would require the backend to run a single coupled simulation loop across all countries rather than independent country loops.

Live data refresh pipeline

Build an automated pipeline that pulls the latest annual data from the World Bank API, IHME, and UNODC on a yearly schedule, re-runs the preprocessing scripts, re-estimates the matrix, validates the

spectral radius, and updates the deployed matrix if all validation checks pass. This would keep the baseline values and structural matrix current without manual intervention.

Country-specific matrix estimation

For countries with sufficient data (e.g. 20+ years), estimate a country-specific 7x7 matrix using only that country's time series data. This would capture structural differences between, for example, Germany's export-driven dynamics and Jamaica's service-dependent dynamics. Countries with insufficient data would fall back to the global matrix. This requires a two-tier matrix architecture in the simulation engine.

User accounts and saved scenarios

Implement authentication and a database layer to allow users to name and save simulation configurations (country selections, scenario, manual shocks, step count), share them via URL, and compare saved configurations side by side.

Long-term (9-24 months)

Expand to 80+ countries

Automate the data pipeline to handle partial-coverage countries by relaxing the requirement that all 7 variables must be present. For countries missing one or two variables, use multiple imputation or structural estimates from similar economies. Target coverage of all World Bank member states.

Nonlinear model extension

Implement a piecewise linear model that switches between two or more A matrices based on detected economic regime (expansion vs contraction, based on GDP growth sign and trend). This captures the empirically documented asymmetry between recession dynamics (fast, sharp) and recovery dynamics (slow, gradual) that the single linear matrix cannot represent.

Natural language scenario specification

Integrate an LLM to translate natural language descriptions into shock parameter vectors. A user types "simulate the effect of a 40% oil price spike combined with US tariff increases" and the LLM maps this to specific shock values for inflation, GDP, and trade-exposed sectors. The translation would be shown transparently to the user before running.

Policy counterfactual comparison

Add a split-screen mode that runs two simulations simultaneously with different configurations and shows side-by-side charts of the diverging trajectories. This would be the primary tool for policy analysis: "what happens under fiscal stimulus versus monetary tightening?" shown as two simultaneous trajectories.